

Construir software con IA

Taller 2: bajo el capó, y tu app en GitHub



TALLER 2

Hoy: entender por dentro

10'	Recapitulación Taller 1 y revisión tarea pendiente	Revisamos lo que se hizo el taller 1 y cuántos trajeron la tarea pendiente
20'	Bajo el capó	Entendiendo lo construido: front, back, bases de datos, APIs, y agentes.
25'	Manos a la obra	publicas tu app
20'	Sigue construyendo	pule lo publicado y prepara tu demo para el cierre del programa.
10'	Cierre	recap y 2-3 muestran sus avances.

Al salir: tu app está en internet, con un link, y tú entiendes qué hay debajo.

Recapitulación Taller 1

1 · TU FICHA — 6 partes

1. Idea en limpio

¿Qué problema resuelve, quién la usa y qué única cosa debe hacer bien?

2. Sí y No

¿Las 3 a 7 cosas que Sí entran, qué espera y qué queda fuera?

3. Recorrido

¿Qué pasos sigue el usuario? Cada paso es una pantalla.

4. Información

¿Qué debe recordar, quién crea, quién mira, hay datos delicados?

5. Tamaño

¿Cuánta gente la usará y qué parte no puede ir lenta?

6. Ejemplo de éxito

¿Cuál es la prueba concreta y verificable de que funciona?

2 · TU PROMPT — 5 partes

Rol

qué experto quieres que sea, y que te explique simple

Contexto

quién eres y qué duele hoy

Tarea

qué debe hacer exactamente

Límites

lo que no debe agregar

Ejemplos

datos de muestra inventados

¿No sabes cómo empezar?
Pídele a la IA que te genere el prompt y ajusta lo que proponga.

3 · TU APP ENCENDIDA

ARCHIVO vs. APP VIVA

Archivo HTML: se ve, pero no hay nada encendido detrás.

App en localhost: viva sobre un mini servidor, encendida solo en tu computador.

El servidor es el garzón: le pides algo y te lo trae. Server, en inglés, es el que sirve.

LO QUE LE PEDISTE

1. «Usa esta ficha como contexto y respétala.»

2. «Usa Next.js para el frente y el fondo.»

3. «Ejecútala en modo desarrollo en mi computador y dime la dirección de localhost. No la publiques en internet.»

LA SEÑAL:
http://localhost:3000

4 · MIENTRAS CONSTRUYES

EL CICLO

Pide

Construye

Revisa

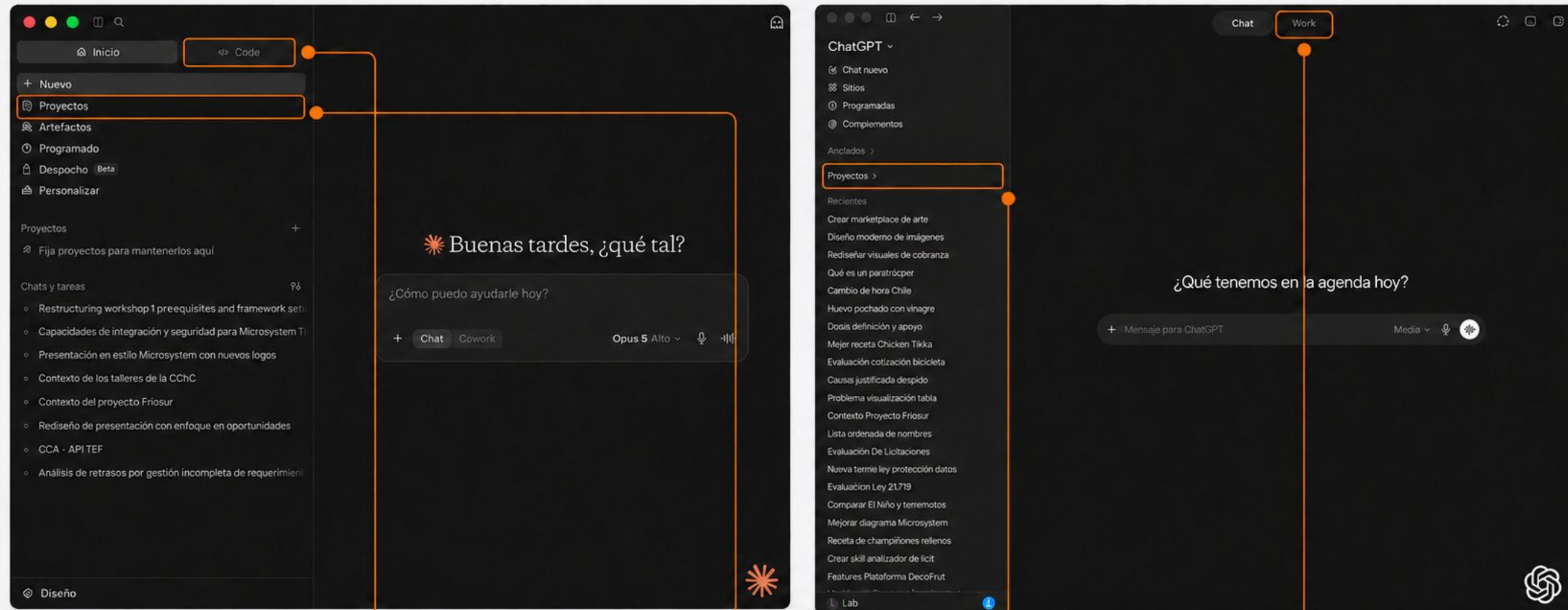
Corrige

- **Revisa contra tu ficha:** si agregó algo fuera de la lista, dile que lo saque.
- **Corrige en concreto:** una cosa a la vez; di qué sí y qué no.
- **Pide de a poco:** un bloque a la vez, no todo junto.
- **Cuida el contexto:** si se confunde, recuérdale tu ficha o parte una conversación nueva.
- **Datos de ejemplo:** nunca información real ni sensible.
- **¿Trabado?** Pregúntale primero a la IA; después levanta la mano.

Tú defines → pides → la IA construye → tú revisas → corriges · listo cuando tu ejemplo de éxito se cumple

CÓMO LEVANTAS TU PROYECTO

Tu interfaz, proyectos y sección de código



2 Code en Claude

Espacio para crear, revisar y modificar código directamente.

1 Proyecto

Reúne chats, archivos e instrucciones de un mismo trabajo y mantiene su contexto organizado.

3 Work en ChatGPT

Modo para ejecutar trabajos completos, incluida la implementación de código.

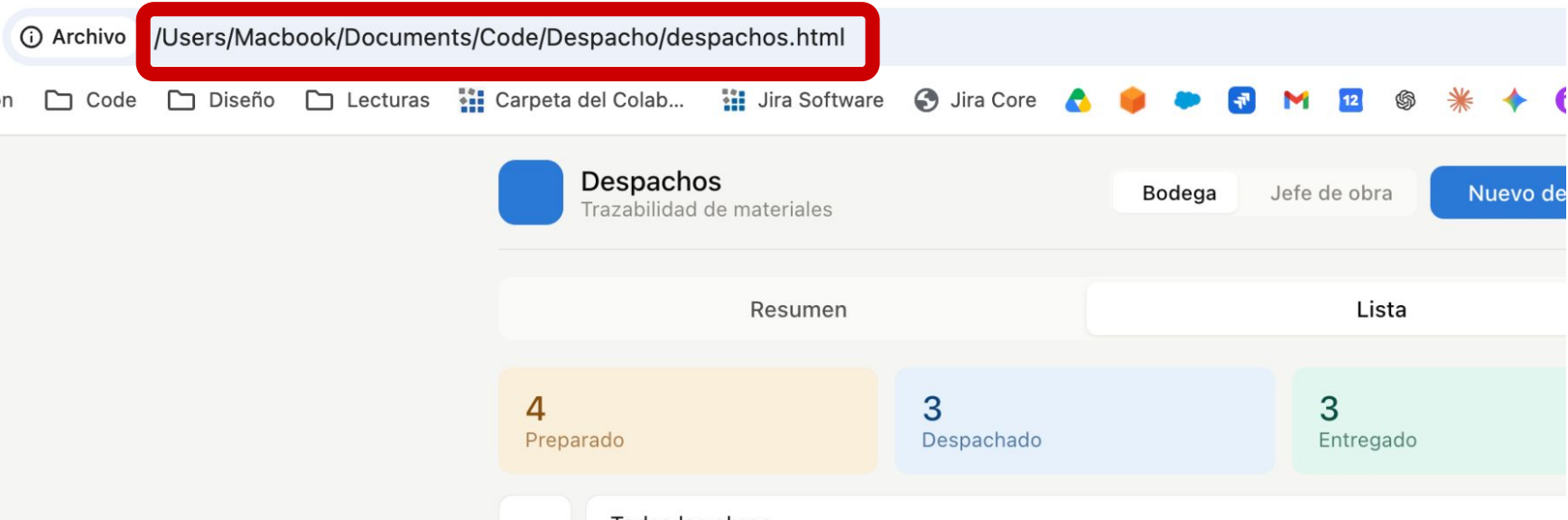
Construye una aplicación, no un archivo plano

La IA escribirá archivos de código por ti. Todos juntos forman tu proyecto: una carpeta, en tu computador o en la plataforma.

Abrir un archivo HTML (Hyper Text Markup Language)

Es un archivo que contiene código y permite que se visualice en el navegador

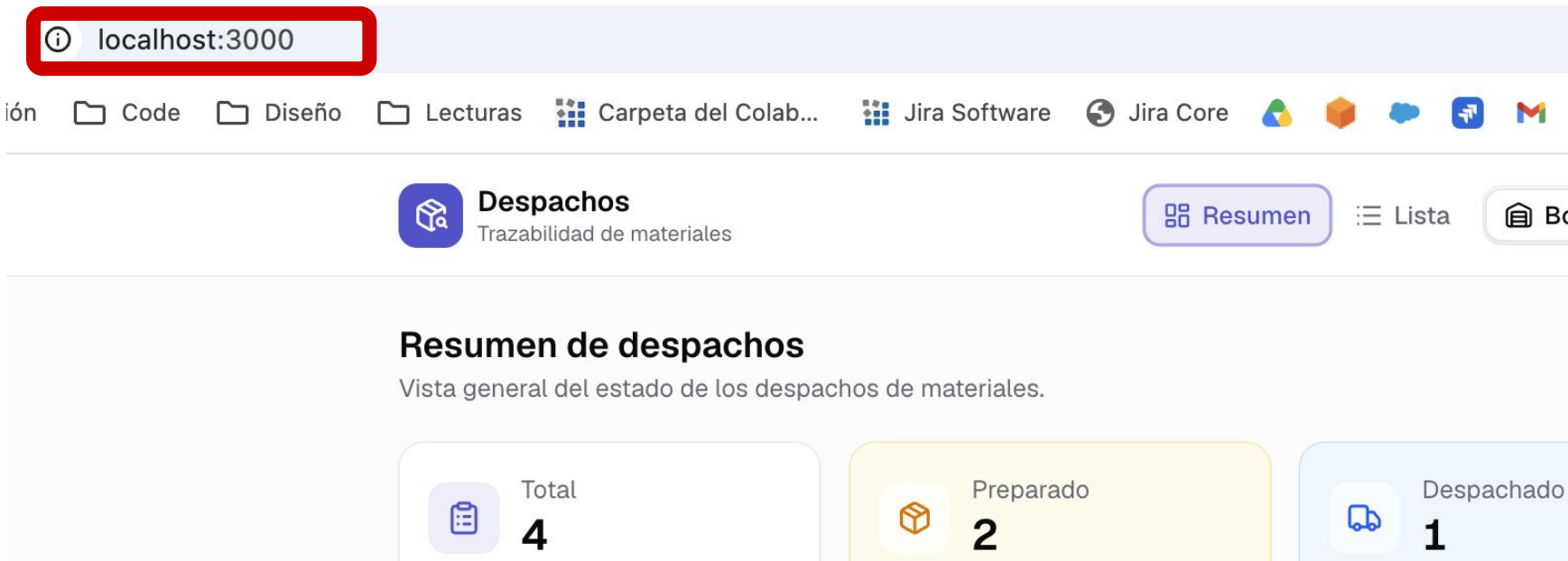
Se ve, pero no hay nada encendido detrás: sin lógica corriendo, sin app de verdad.



“Corre en tu computador”

Un mini servidor enciende tu app completa en tu máquina (eso es localhost).

Es la app viva sobre un servidor. Solo tú la ves, porque está encendida en tu computador.



Conceptos Básicos para el Desarrollo de Software con IA

BAJO EL CAPÓ

La programación con IA es un proceso cada vez más automático



"Software is eating the world. AI is eating software." — Marc Andreessen (adaptado)

¿Por qué la IA es tan buena generando código?



Lenguaje formal

El código tiene sintaxis estricta y reglas sin ambigüedad.

Los modelos aprenden esas reglas con facilidad.

500M+

Datos de entrenamiento

GitHub tiene más de 500 millones de repositorios.

Casi todo patrón de código ya fue escrito miles de veces.



Verificable al instante

A diferencia de un ensayo, el código se corre y se sabe de inmediato si funciona.

Ciclos de aprendizaje muy rápidos.

"The hottest new programming language is English." — Andrej Karpathy, 2023

BAJO EL CAPÓ

La programación: el caso de uso #1 de la IA generativa

☞ Uso de IA por categoría profesional

Porcentaje de conversaciones con Claude según el Anthropic Economic Index.



Fuente del gráfico: Anthropic Economic Index — categorías ocupacionales con mayor uso de Claude.

84%

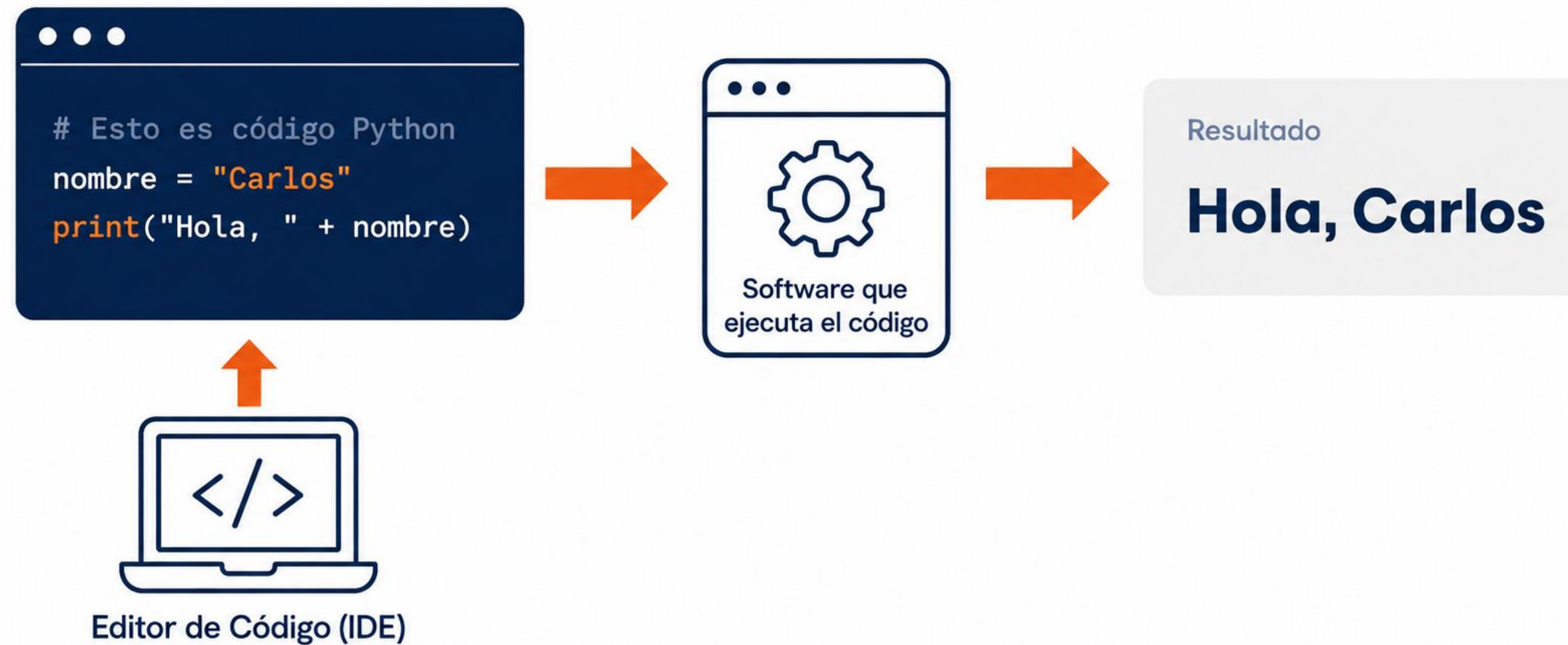
de los desarrolladores
ya usan o planean usar
IA en su flujo de trabajo

*Stack Overflow Developer
Survey, 2025*

BAJO EL CAPÓ

¿Qué es código?

Instrucciones escritas en un lenguaje que tanto humanos como computadoras pueden entender.



¿Qué es código?

El código nunca está terminado: se mejora, se corrige y a veces lo trabajan varias personas. Por eso necesitamos gestionar sus versiones.

```
# Historial de versiones  
v1  Primera versión de la app  
v2  Arreglé el login  
v3  Agregué modo oscuro      · Ana  
v4  Mejoré la velocidad      · Luis
```

En la nube

GitHub

Un **repositorio** (o "repo") es la carpeta que guarda todo el código de un proyecto junto con su historial de cambios.

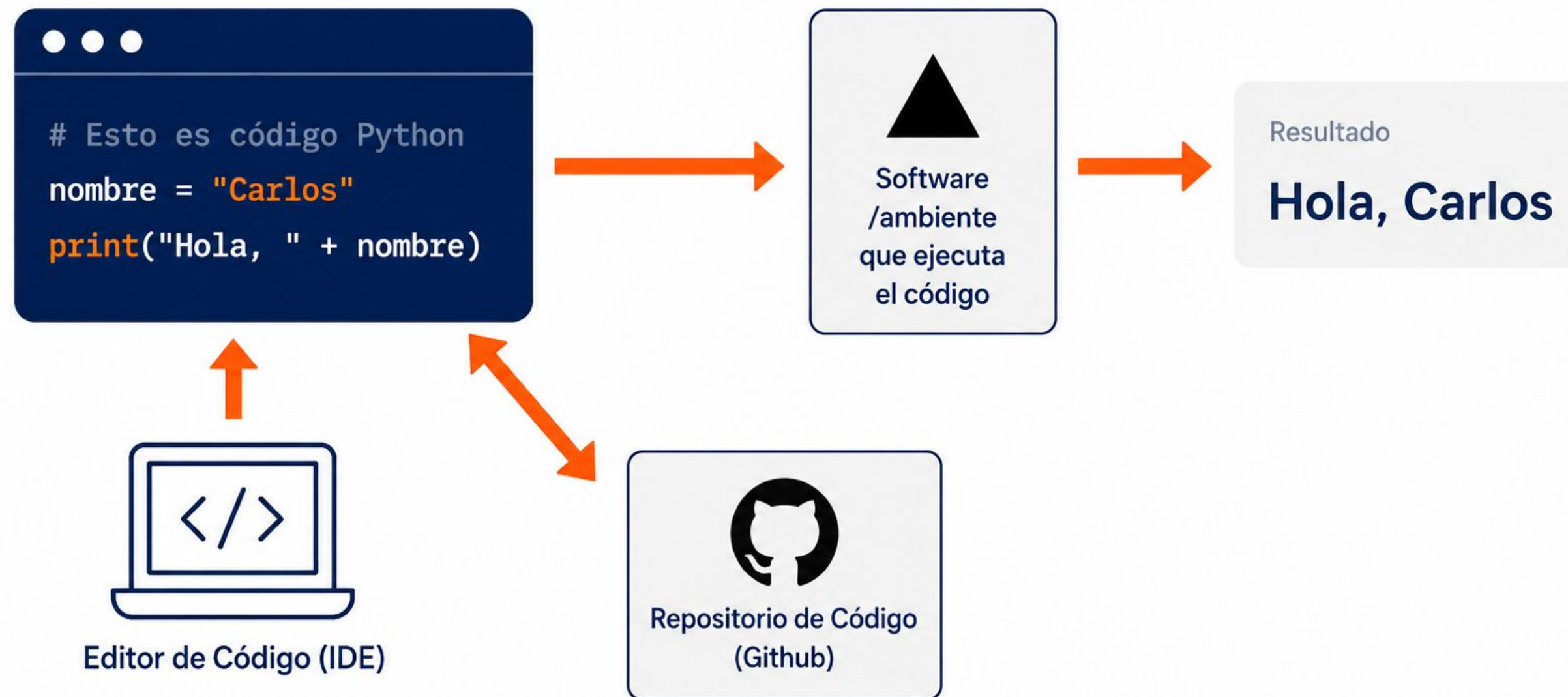
Github permite volver a una versión anterior si algo se rompe

También permite generar múltiples versiones del repo (ramas) para que varias personas pueden trabajar cada una en una rama sin pisarse.

BAJO EL CAPÓ

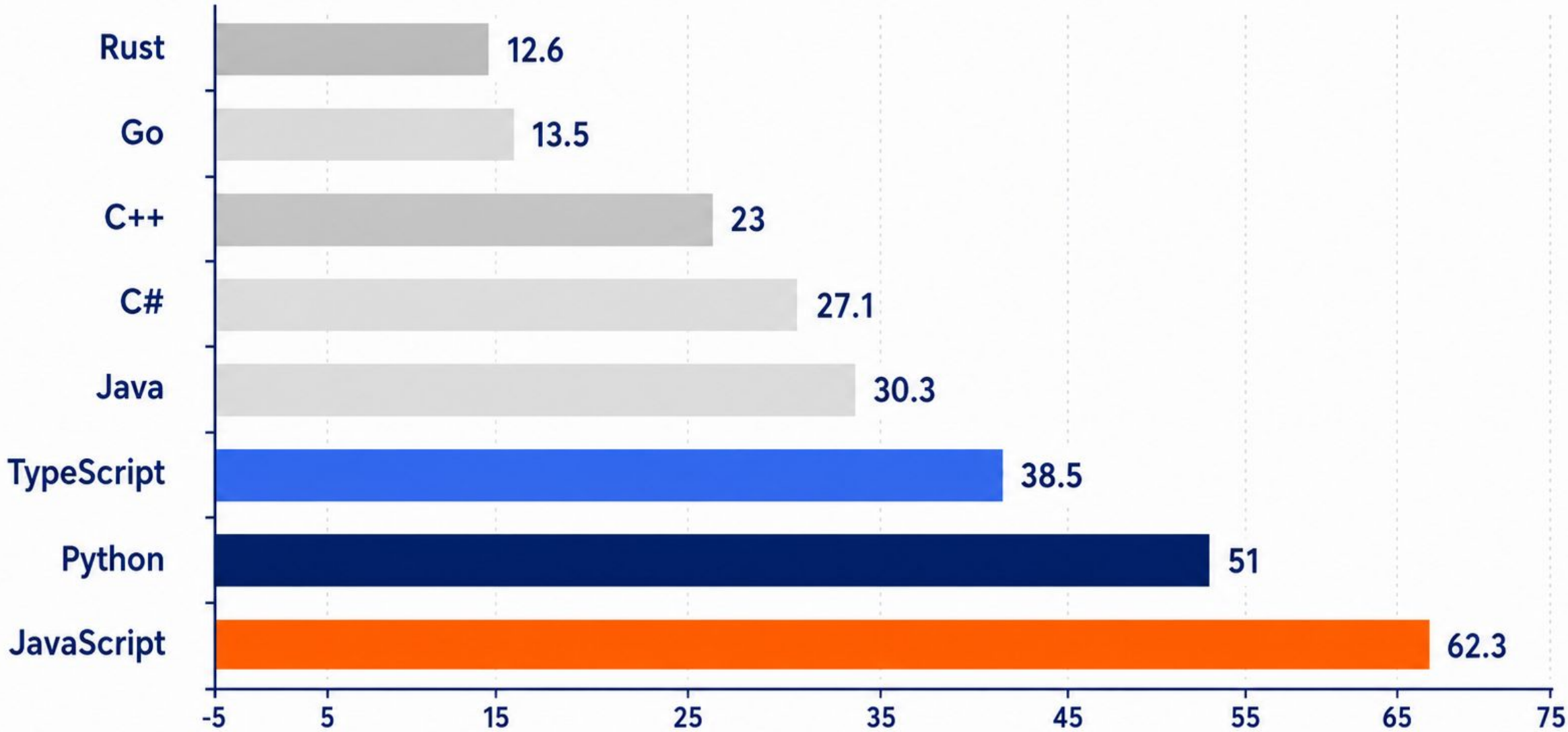
¿Qué es código?

Instrucciones escritas en un lenguaje que tanto humanos como computadoras pueden entender.



Los lenguajes de programación (o codificación) más usados

Lenguajes de programación más usados



Fuente: Stack Overflow Developer Survey 2024 · % de desarrolladores que lo usan activamente

JavaScript

#1 del mundo
12 años consecutivos

Python

Dominante en IA / ML
y scripting

¿Importa saber programar (codificar)?

"El 99% del tiempo no estás escribiendo código sino orquestando agentes y actuando como un supervisor."

Andrej Karpathy

co-fundador OpenAI · Feb
2026

*"El 30% del código en nuestros repos ya lo escribe IA.
Y ese número va a seguir subiendo."*

Satya Nadella

CEO Microsoft · Build 2025

La barrera bajó. Pero construir bien sigue siendo difícil.

→ Los 5 desafíos reales

Conceptos Necesarios para construir software

1 Definir los requerimientos

El modelo construye exactamente lo que describes. Descripción vaga = resultado vago. La claridad es la diferencia entre 3 iteraciones y 30.

2 Arquitectura

Más usuarios, más datos, más integraciones y más confiabilidad exigen diseñar una buena arquitectura de software.

3 Calidad

El código puede verse correcto y estar roto. Con IA, el código luce plausible incluso con bugs. La respuesta son los tests.

4 Seguridad

El código generado rápido también amplía la superficie de riesgo: credenciales expuestas, dependencias sin revisar y permisos mal definidos.

5 Software versus agentes

La siguiente frontera: agentes que construyen otros agentes. Lo que no cambia: alguien tiene que saber si el resultado está correcto.

Conceptos Necesarios para construir software

1 Definir los requerimientos

El modelo construye exactamente lo que describes. Descripción vaga = resultado vago. La claridad es la diferencia entre 3 iteraciones y 30.

2 Arquitectura

Más usuarios, más datos, más integraciones y más confiabilidad exigen diseñar una buena arquitectura de software.

3 Calidad

El código puede verse correcto y estar roto. Con IA, el código luce plausible incluso con bugs. La respuesta son los tests.

4 Seguridad

El código generado rápido también amplía la superficie de riesgo: credenciales expuestas, dependencias sin revisar y permisos mal definidos.

5 Software versus agentes

La siguiente frontera: agentes que construyen otros agentes. Lo que no cambia: alguien tiene que saber si el resultado está correcto.

Tipos de software: del más simple al más complejo

01

HTML

Estructura, sin lógica

Define qué aparece en el browser. Un archivo .html ya es una página web. Sin servidor, sin instalación.

Ej.

Home page de una empresa — solo muestra texto e imágenes. Como un Word con otro formato que leen los navegadores.

02

Script

Automatiza una tarea

Hace una tarea acotada, específica, típicamente para automatizar. Corre una vez, hace algo, termina. Sin interfaz.

Ej.

Una macro que corre en un Excel.

Un programa en Python para leer datos de una página web.

03

Aplicación

Múltiples piezas coordinadas

Interactividad y persistencia de datos, puede soportar múltiples usuarios. Siempre corriendo.

Ej.

Un software como el que construyeron en este curso.

04

Agente

Decide cómo actuar en forma autónoma

Un software que invoca modelos LLM para decidir cómo actuar. Tiene herramientas, memoria y un loop de decisión.

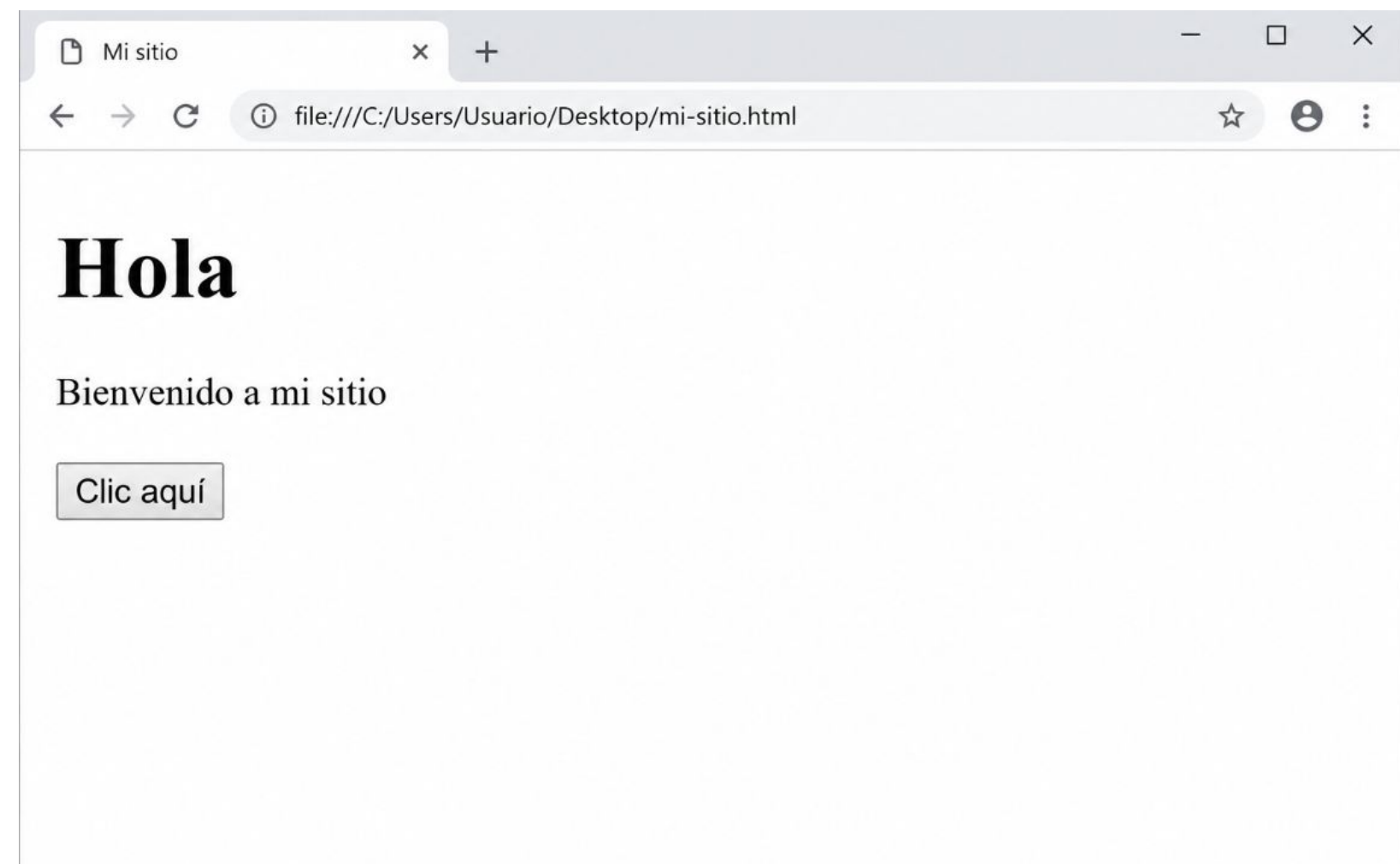
Ej.

Claude Code: le dices el objetivo y él decide qué archivos leer, qué código escribir y cuándo terminar.

¿Qué es el HTML?

Tipo de Código que lee navegador (chrome, safari) lee y genera una página web.

```
<!-- Esto es HTML -->  
<h1>Hola</h1>  
<p>Bienvenido a mi sitio</p>  
<button>Clic aquí</button>
```

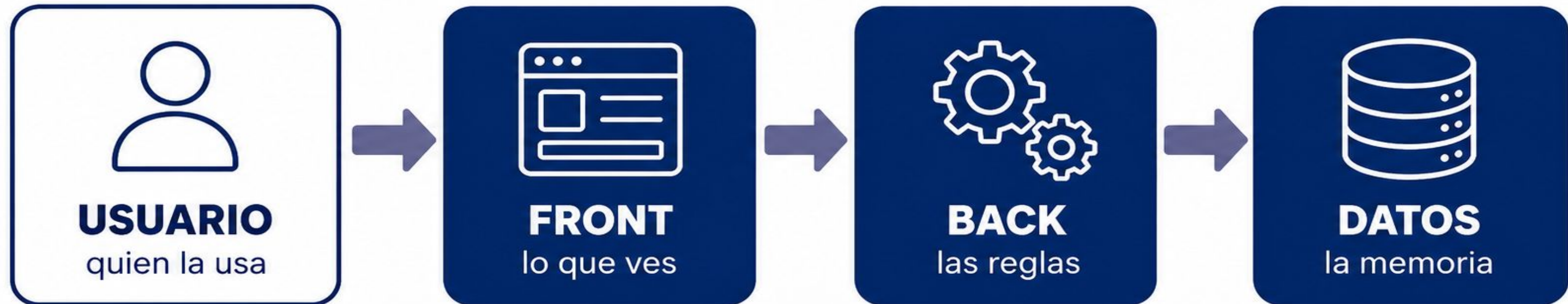


Arquitectura básica de una Aplicación



AUTENTICACIÓN

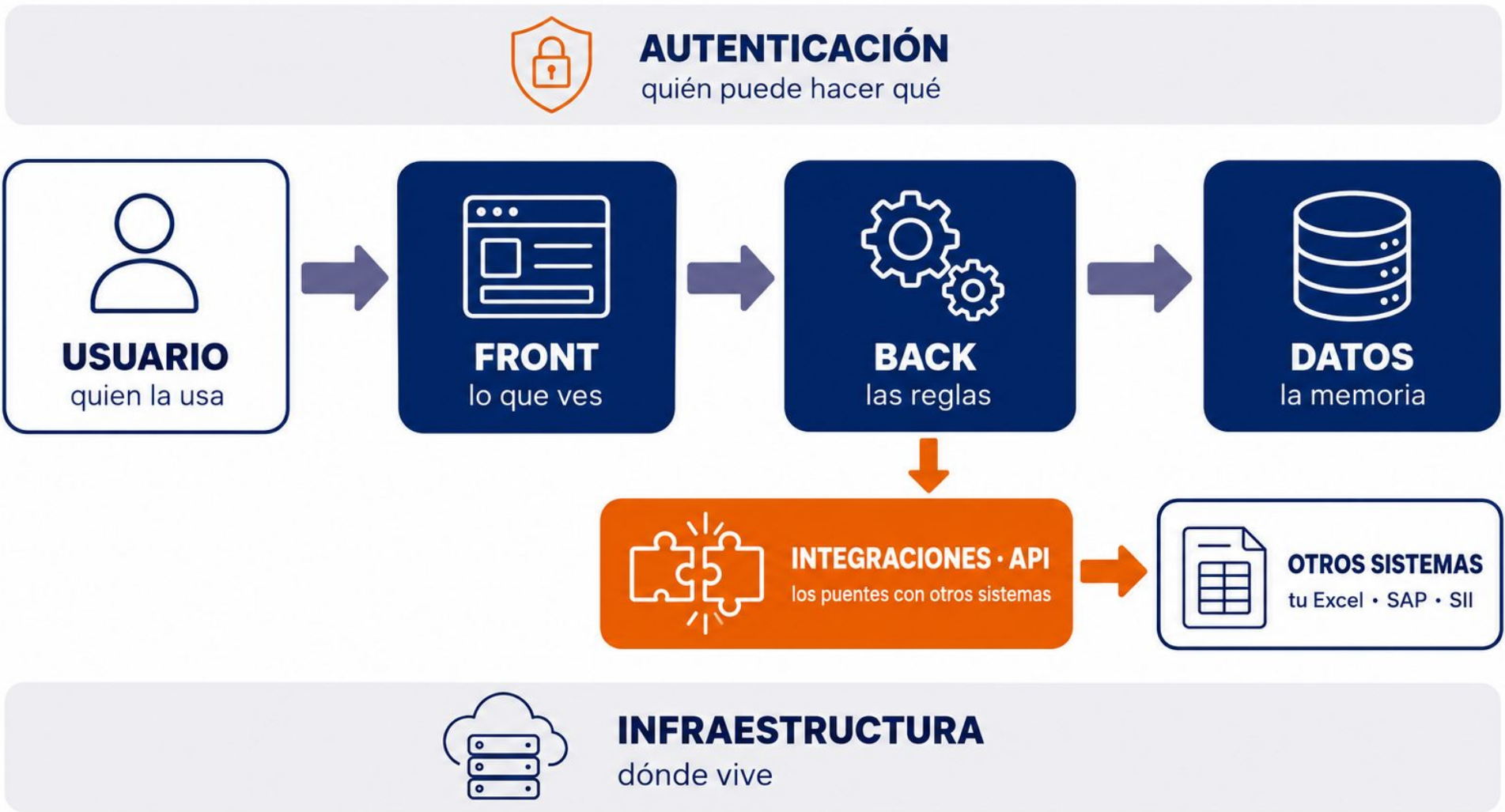
quién puede hacer qué



INFRAESTRUCTURA

dónde vive

API's: Enchufes a otros programas que trabajan en mi aplicación



Ejemplos de API

Pagos

Cobrar con tarjeta de forma segura

Mapas y ubicación

Mostrar direcciones y rutas

Mensajería

Enviar emails y SMS

Inteligencia artificial

Llamar a un LLM para generar texto, imágenes y respuestas

Obtener datos

Obtener la UF diraiia de la API de la CMF

Los datos son el cimiento

El modelo de datos es la decisión más difícil de deshacer.

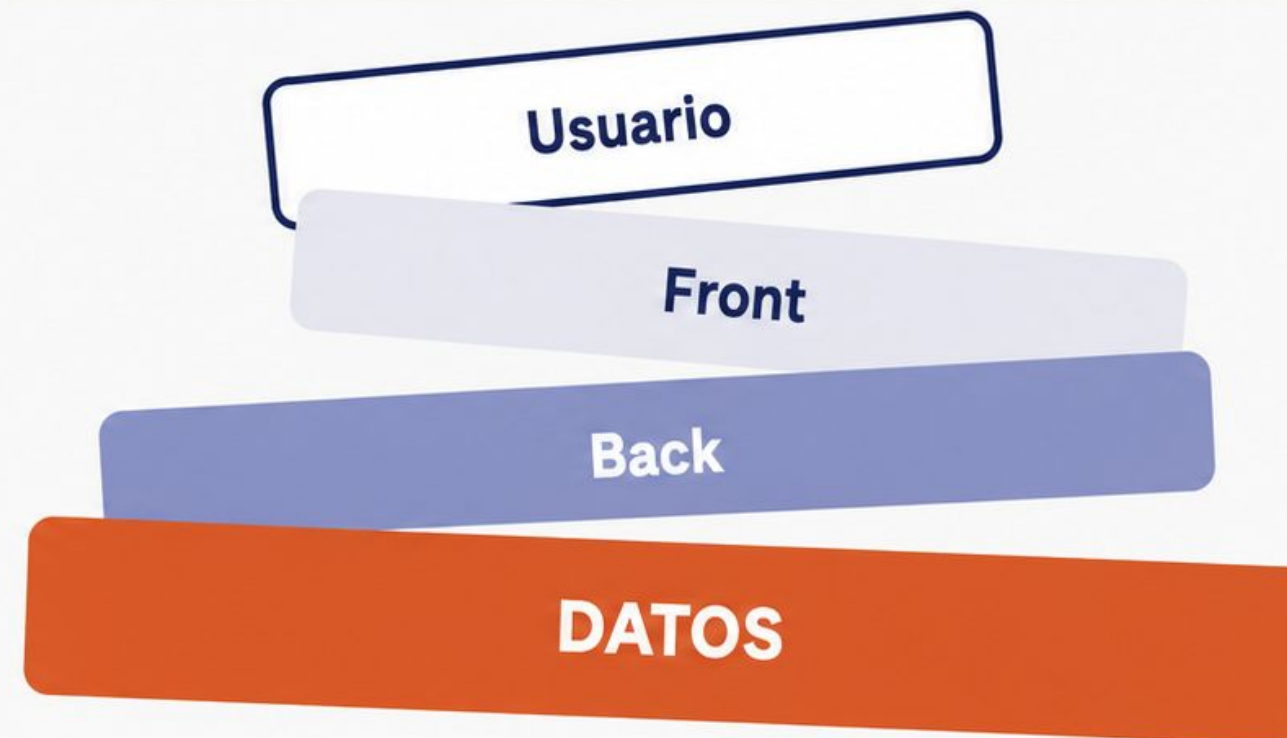
✓ Modelo bien pensado

Cimiento firme: cada piso nuevo simplemente se apoya.



✗ Modelo improvisado

Cimiento chueco: cada cambio pelea con la base.



Si el modelo está bien pensado, el resto fluye. **Si está mal, todo duele.**

Las aplicaciones hoy se construyen sobre frameworks

No tienes que programar todo desde cero. Los frameworks tienen armada la estructura de cada capa: rutas, base de datos, autenticación, renderizado. Tú agregas la lógica de negocio.

FRONTEND

Lo que el usuario ve en el browser

React, Vue.js, **Next.js** (parte UI)

BACKEND

La lógica, las reglas, la API

Express, FastAPI, Django, **Next.js** (parte server)

FULLSTACK

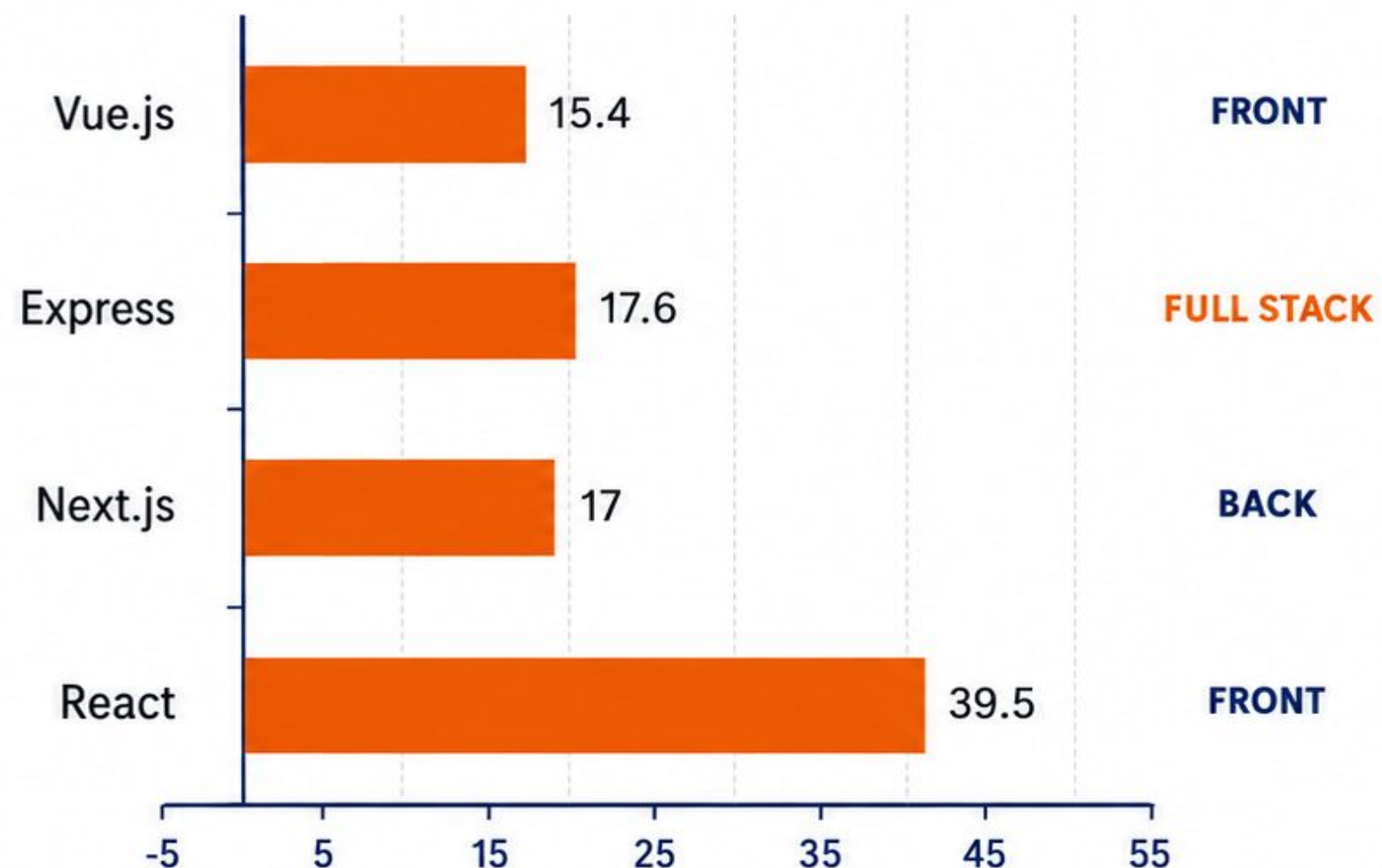
Frontend + backend en un solo proyecto

Next.js, Nuxt.js, Django (con templates)

El framework cambia. Los principios son los mismos en cualquier stack.

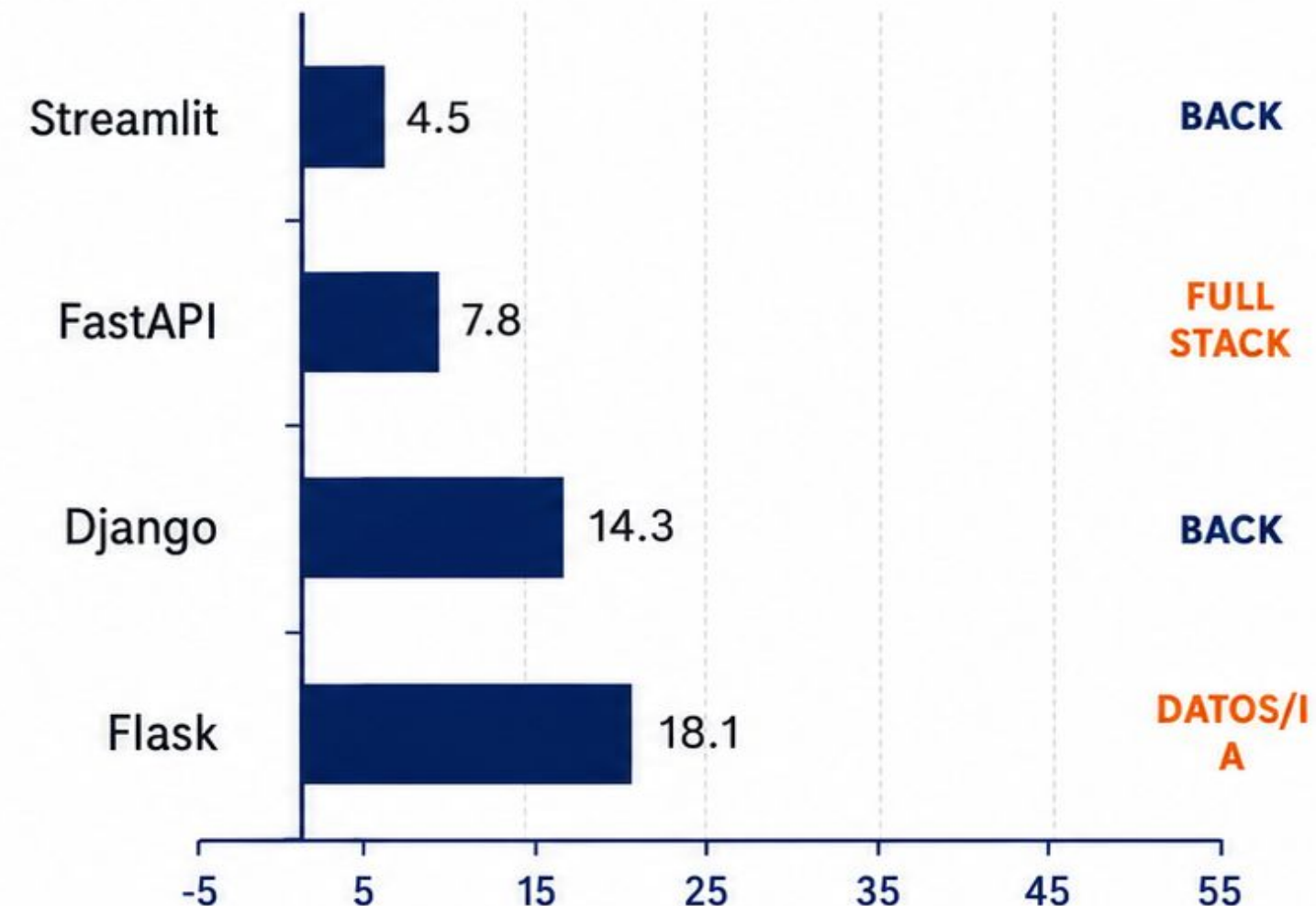
Frameworks más usados

JavaScript / TypeScript



⚠ Node.js no es un framework — es el entorno de ejecución de JavaScript fuera del browser. Express y Next.js corren sobre él.

Python



Conceptos Necesarios para construir software

1 Definir los requerimientos

El modelo construye exactamente lo que describes. Descripción vaga = resultado vago. La claridad es la diferencia entre 3 iteraciones y 30.

2 Arquitectura

Más usuarios, más datos, más integraciones y más confiabilidad exigen diseñar una buena arquitectura de software.

3 Calidad

El código puede verse correcto y estar roto. Con IA, el código luce plausible incluso con bugs. La respuesta son los tests.

4 Seguridad

El código generado rápido también amplía la superficie de riesgo: credenciales expuestas, dependencias sin revisar y permisos mal definidos.

5 Software versus agentes

La siguiente frontera: agentes que construyen otros agentes. Lo que no cambia: alguien tiene que saber si el resultado está correcto.

Vibe coding no necesariamente produce software con menos errores y problemas

1.7x

más problemas críticos

Código co-escrito con IA tiene 1.7x más issues mayores que código humano.

Vulnerabilidades de seguridad: 2.74x más.

CodeRabbit — análisis de 470 PRs open source, 2025

45%

del código tiene vulnerabilidades

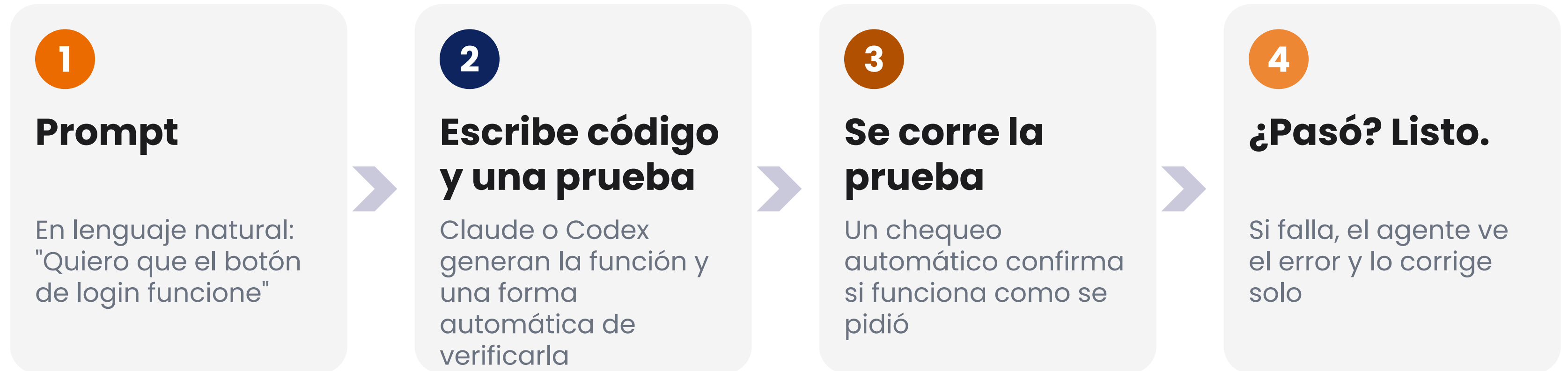
El 45% del código generado por IA introduce al menos una vulnerabilidad del OWASP Top 10 en producción.

Veracode GenAI Code Security Report — 100+ LLMs, 2025

La IA acelera escribir código, pero podría producir software de muy mala calidad.

LOS 4 DESAFÍOS DE CONSTRUIR SOFTWARE: GESTIONAR LA CALIDAD

Vibe coding con pruebas



Si la prueba falla:

el agente corrige el código y vuelve a probar, hasta que todo funcione.

LOS 4 DESAFÍOS DE CONSTRUIR SOFTWARE: GESTIONAR LA CALIDAD

Testeo en una aplicación

Un test es una verificación automática de que el código hace lo que se espera.

Unit test

Una pieza pequeña, en aislamiento

"Dado este descuento del 20%, ¿el precio calculado es correcto?"

Integration test

Dos partes trabajando juntas

"Cuando el usuario envía el formulario, ¿el dato llega bien a la DB?"

End-to-end (E2E)

Un usuario real usando la app completa

"Entro, creo un contacto, busco por nombre, veo el resultado correcto"

Con IA generando código, los tests importan más, no menos: el código luce plausible incluso cuando tiene bugs.

Conceptos Necesarios para construir software

1 Definir los requerimientos

El modelo construye exactamente lo que describes. Descripción vaga = resultado vago. La claridad es la diferencia entre 3 iteraciones y 30.

2 Arquitectura

Más usuarios, más datos, más integraciones y más confiabilidad exigen diseñar una buena arquitectura de software.

3 Calidad

El código puede verse correcto y estar roto. Con IA, el código luce plausible incluso con bugs. La respuesta son los tests.

4 Seguridad

El código generado rápido también amplía la superficie de riesgo: credenciales expuestas, dependencias sin revisar y permisos mal definidos.

5 Software versus agentes

La siguiente frontera: agentes que construyen otros agentes. Lo que no cambia: alguien tiene que saber si el resultado está correcto.

Al Ejército de Chile le robaron 360.000 documentos secretos

Qué fue del hackeo y subasta de información secreta del Ejército de 2023



Documentos del Ejército filtrados por Rhysida.

🎯 En mayo, un grupo de hackers robó y subastó información secreta del Ejército. *Interferencia* accedió al 30% de los archivos que fueron publicados como prueba de la venta, detectando documentos altamente comprometedores para la seguridad nacional. Desde entonces, las instituciones guardaron silencio y se culpó a un cabo. Los datos siguen disponibles en la dark web. Aquí repasamos nuestra cobertura.

- 27 may 2023 Rhysida entra a la red del Ejército.
- 29 may 2023 La institución confirma la brecha y aísla la red.
- 10 jun 2023 Rhysida publica al Ejército en su leak site.
Empieza la extorsión pública.
- jun 2023 Subasta los documentos. Nadie paga.
"Data hunters, enjoy": libera 360.000 archivos gratis.
- después Un cabo del Ejército es detenido y formalizado.
Defensa guarda silencio; la ANI declara en secreto.

360.000

documentos publicados

30%

de lo realmente robado

318

credenciales expuestas

US\$ 0

pagados de rescate

Todo esto es público, gratis y lo puedes revisar hoy

01

ransomware.live

RESPONDE: A QUIÉN

Espeja los "leak sites" que los grupos publican en la dark web.

- Ejercito de Chile · Rhysida
- 10-jun-2023 · Defensa / Gobierno
- VPN Fortinet expuesta · CVE-2022-40684



Ficha de la víctima
ransomware.live

02

MITRE ATT&CK

RESPONDE: CÓMO

El lenguaje común para describir tácticas y técnicas de ataque.

- T1078 Valid Accounts · VPN sin MFA
- T1219 AnyDesk · T1021 RDP y SSH
- T1070.001 · borran los logs



Perfil Rhysida
CISA AA23-319A

03

Exploit-DB

RESPONDE: CON QUÉ

El repositorio público de código de explotación funcional.

- EDB-52239 · CVE-2022-40684
- Inyecta llave SSH y toma el firewall
- EDB-49071 · Zerologon



Exploit verificado
exploit-db.com/52239

La falla de Fortinet es de octubre de 2022. El ataque, de mayo de 2023. **No falló la tecnología: faltó parchar y activar MFA.**

Control de acceso y perfilamiento de usuarios

Definir desde el inicio quién entra, qué puede hacer y con qué permisos.

Autenticación

¿Quién eres?

Verificar la identidad: contraseña + segundo factor, o entrar con Google/GitHub. Las contraseñas nunca se guardan en texto plano.

Permisos

¿Qué puedes hacer?

Cada usuario tiene un rol (administrador, editor, solo lectura) que le permite realizar acciones en la aplicación.

Administración de Usuarios y Permisos

Administración

Las aplicaciones tienen secciones para administrar usuario, roles y permisos.

Pide el sistema de roles y permisos desde el primer día: agregarlo al final cuesta mucho más.

LOS 5 DESAFÍOS DE CONSTRUIR SOFTWARE: SEGURIDAD

Buenas prácticas de Seguridad

Una aplicación puede funcionar bien y aun así quedar expuesta al publicarla.

HTTPS y secretos

Cifrar y no exponer claves

Todo el tráfico viaja cifrado con HTTPS. Las claves y contraseñas viven en variables de entorno, nunca dentro del código.

Sin exposición pública

Seguridad perimetral

La base de datos y los servicios internos no deben tener IP ni URL pública: solo la aplicación llega a ellos, por una red privada y detrás del cortafuegos.

Límites y monitoreo

Controlar, registrar, actualizar

Limitar cuántas peticiones se aceptan por minuto, registrar quién hizo qué y mantener las librerías al día.

Antes de publicar, pide a la IA un checklist de seguridad para tu stack (referencia: OWASP Top 10).

Conceptos Necesarios para construir software

1 Definir los requerimientos

El modelo construye exactamente lo que describes. Descripción vaga = resultado vago. La claridad es la diferencia entre 3 iteraciones y 30.

2 Arquitectura

Más usuarios, más datos, más integraciones y más confiabilidad exigen diseñar una buena arquitectura de software.

3 Calidad

El código puede verse correcto y estar roto. Con IA, el código luce plausible incluso con bugs. La respuesta son los tests.

4 Seguridad

El código generado rápido también amplía la superficie de riesgo: credenciales expuestas, dependencias sin revisar y permisos mal definidos.

5 Software versus agentes

La siguiente frontera: agentes que construyen otros agentes. Lo que no cambia: alguien tiene que saber si el resultado está correcto.

Los agentes son software

Ejemplo: agente que agenda una reunión

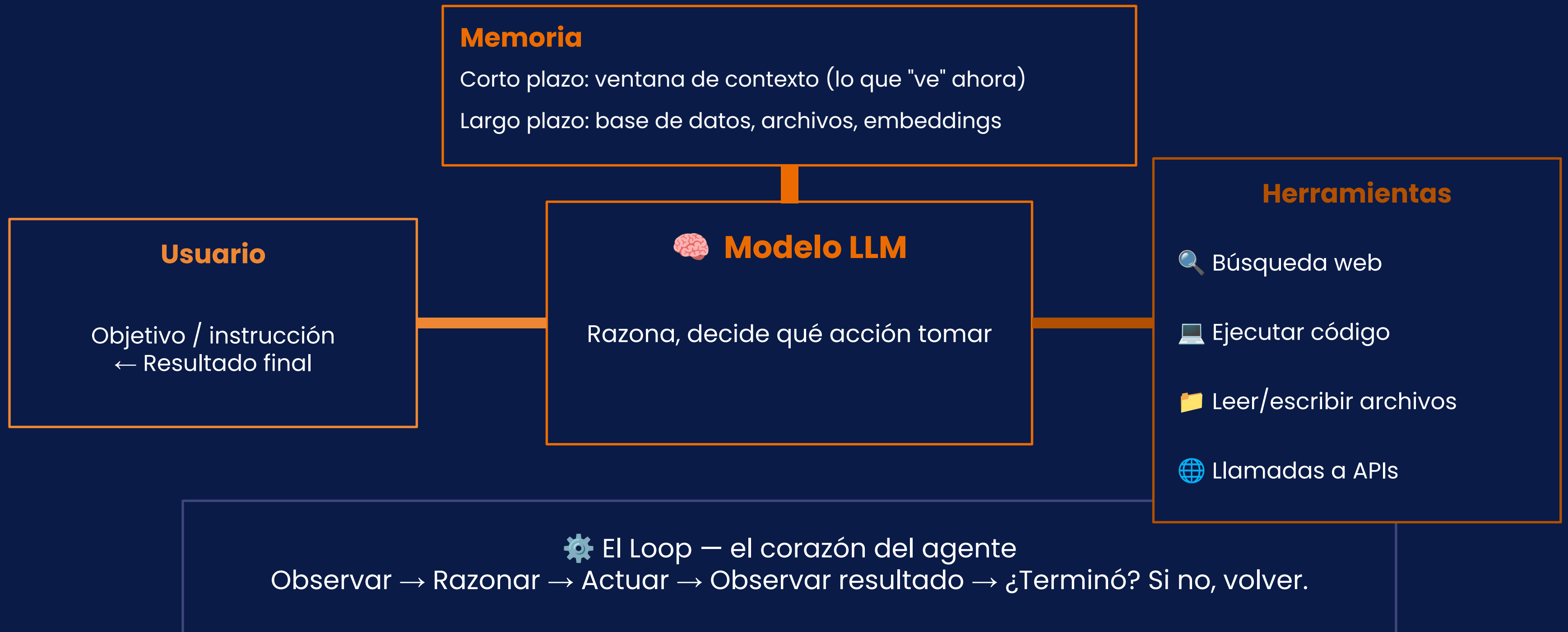


↺ Si no terminó, vuelve a decidir



La diferencia con un script: el agente decide qué pasos dar. Pero sigue siendo código el que orchestra esa decisión.

¿En qué consiste un agente?



Agentes avanzados suman: planificación previa (Planning) y autocrítica del output (Reflection).

LOS 4 DESAFÍOS DE CONSTRUIR SOFTWARE: GESTIONAR LA CALIDAD

Principales Frameworks para construir agentes

OpenAI**Agents SDK**

SDK oficial de OpenAI. Define agentes con instrucciones, herramientas y handoffs entre agentes.

Python / TypeScript**Anthropic****Claude Agent SDK + MCP**

SDK de Anthropic más Model Context Protocol: estándar abierto para conectar herramientas a cualquier modelo.

Python / TypeScript**Google****Vertex AI + ADK**

Agent Development Kit de Google. Integra Gemini con búsqueda, BigQuery y servicios de Google Cloud.

Python**Microsoft****AutoGen + Azure AI**

AutoGen orquesta conversaciones multi-agente. Azure AI Foundry lo despliega en infraestructura enterprise.

Python**LangChain****LangGraph**

Popular en la comunidad open source.

Python / JS**Open source****CrewAI**

Define equipos de agentes con roles. Fácil de empezar. Ideal para prototipos multi-agente rápidos.

Python

Publica en Github



¿Y si rompes lo que ya funciona?

Lo que tienes te gusta y funciona. Quieres guardarlo —y si al modificar algo se rompe, poder volver al punto anterior.

QUÉ SON

Git: el sistema que registra cada cambio y te deja volver a cualquier punto.

GitHub: plataforma que permite almacenar tu carpeta con versiones, en internet: el proyecto y todo su historial, en la web.

QUÉ GANAS

- **Checkpoints:** vuelve a la última versión que funcionaba.
- **Respaldo:** tu proyecto deja de vivir solo en tu computador.
- **Desde cualquier parte:** lo abres y sigues donde estés.
- **Trabajo en equipo:** varias personas, el mismo proyecto, sin pisarse.
- **Quién hizo qué:** cada cambio queda con autor y fecha: control total.
- **Tu contexto viaja:** tu CLAUDE.md / AGENTS.md acompaña al proyecto.



LA ANALOGÍA: CHECKPOINT

Como un juego difícil: cuando pierdes, no partes de cero —vuelves al último checkpoint y sigues desde ahí. **Cada versión guardada de tu proyecto es un checkpoint.**

Y es el requisito para publicar: la próxima sesión, tomaremos el proyecto **desde GitHub** y lo publicaremos en internet con un link.



GitHub

Sube tu proyecto a GitHub

CÓMO SE LO PIDES A LA IA

"Sube este proyecto a un repositorio nuevo en mi cuenta de GitHub y guíame paso a paso. Si algo falla, explícame qué hacer."

QUÉ SE PUBLICA

Un **repositorio** es la carpeta de tu proyecto en GitHub, con todos sus archivos y su historial de versiones.

Se sube tu código (no una foto): GitHub lo muestra como una lista de archivos que se abren, y guarda cada versión.

LO QUE TE VA A PEDIR

- **Iniciar sesión en GitHub:** autorizar que tu equipo se conecte a tu cuenta.
- **Un nombre para el repositorio:** el de tu proyecto sirve.
- **Público o privado:** si es que quieres que cualquier persona acceda o solo los que tú les brindas acceso (**privado**).

DE AHORA EN ADELANTE: CADA AVANCE, A GITHUB

1 • Commit: sella tu versión

Cuando tu nueva versión te gusta y funciona, la marcas como un checkpoint: "guarda esta versión".

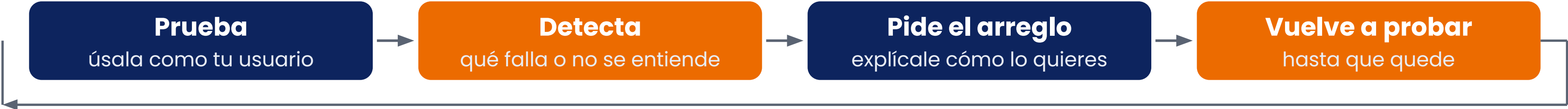


2 • Push: la sube a GitHub

Publicas ese checkpoint en internet: "sube estos cambios a GitHub". Ahí queda a salvo, fuera de tu computador.

Perfecciona tu app

NUEVO CICLO



QUÉ REVISAR

- ☐ **El flujo completo:** de punta a punta: tu ejemplo de éxito se cumple.
- ☐ **Cada botón hace algo:** Clic en todo: ¿hay botones muertos, links que no llevan a ninguna parte, acciones que no pasan nada?
- ☐ **Los caminos de vuelta:** No solo crear: ¿puedes editar, cancelar a mitad de camino, borrar algo que ingresaste mal? Las apps de la IA suelen tener solo el camino feliz.
- ☐ **Campos vacíos y datos absurdos:** ¿deja guardar sin material? ¿cantidad -5?
- ☐ **La prueba del colega:** ¿Alguien de tu empresa la entendería sin que tú se la expliques? Revisa etiquetas y nombres: que digan "Obra de destino", no "Item 2". Que hablen tu idioma, el del negocio.
- ☐ **Con datos realistas:** cárgale 10–15 registros, ¿Qué ocurre?
- ☐ **En el celular:** haz clic derecho y apreta "inspeccionar", prueba los diferentes tamaños de pantalla, ¿Cómo se ve? (tip: esto se llama **responsividad**).
- ☐ **Recarga la página:** ¿qué se pierde?

¿Todo funciona? agrega nuevas funcionalidades

Dashboard

Buscador y Filtros

Mantenedor

Menú Principal